PDC Exam Cram Guide (Chill Version)

Exam: 50% theory + 50% numerical. Focus = Assignment 3 & 4. Don't skip ANY numerical.

Read this top to bottom. I explain everything like you've never seen it. The numericals are first because they're easy marks, learn the method once and reuse it.

PART A - THE NUMERICALS (do these first, they're free marks)

There are basically **4 numerical types** in your sheets. Learn the recipe for each.

NUMERICAL TYPE 1 - NTP (Clock Sync): Delay & Offset

This is the **most likely numerical** (it appears TWICE in your sheet: London/Singapore and New York/Tokyo).

The story (ELI5)

A client computer wants to set its clock using a server's clock. They send messages back and forth. We record **4 timestamps**:

Symbol	Meaning
t1	Client **sends** request
t2	Server **receives** request
t3	Server **sends** reply
t4	Client **receives** reply

The 2 formulas (MEMORIZE THESE)

```
Round-trip delay:   d = (t4 - t1) - (t3 - t2)
Clock offset:       o = [ (t2 - t1) + (t3 - t4) ] / 2
```

What they mean: - **d (delay)** = total time the message spent travelling on the network (minus the time the server spent thinking). It's the "ping" basically. - **o (offset)** = how far off the client's clock is from the server's clock. - If **o is POSITIVE** then client is **BEHIND** the server (client clock is slow, needs to move forward). - If **o is NEGATIVE** then client is **AHEAD** of the server (client clock is fast).

Worked Example 1 - London server / Singapore client

Given: t1 = 1000, t2 = 1030, t3 = 1030, t4 = 1020 (all ms)

Delay:

```
d = (t4 - t1) - (t3 - t2)
d = (1020 - 1000) - (1030 - 1030)
d = 20 - 0
d = 20 ms
```

Offset:

```
o = [ (t2 - t1) + (t3 - t4) ] / 2
o = [ (1030 - 1000) + (1030 - 1020) ] / 2
o = [ 30 + 10 ] / 2
o = 40 / 2
o = +20 ms
```

Answer: Delay = **20 ms**, Offset = **+20 ms**, so Singapore client is **20 ms BEHIND** London. To fix, Singapore adds 20 ms to its clock.

Worked Example 2 - New York server / Tokyo client

Given: t1 = 2500, t2 = 2545, t3 = 2550, t4 = 2520 (all ms)

Delay:

```
d = (2520 - 2500) - (2550 - 2545)
d = 20 - 5
d = 15 ms
```

Offset:

```
o = [ (2545 - 2500) + (2550 - 2520) ] / 2
o = [ 45 + 30 ] / 2
o = 75 / 2
o = +37.5 ms
```

Answer: 1. Delay d = **15 ms** 2. Offset o = **+37.5 ms** 3. Since o is positive, Tokyo client is **BEHIND** New York server, by **37.5 ms**.

> Trick to remember the offset sign: Positive = "client is late/behind." Just plug into the formula and read the sign.

Bonus - Cristian's Algorithm (simpler cousin, just in case)

If they only give you when the request was sent (T0) and reply received (T1) and the server's time (T_server):

```
RTT (round trip time) = T1 - T0
Estimated correct time = T_server + RTT/2
```

The idea: assume the reply took half the round-trip to come back, so add half the ping to the server's time. **NTP is just the smarter version of this.**

NUMERICAL TYPE 2 - HDFS Storage Planning

This is the big one (Q2 on your numerical sheet, 4 parts). Looks scary, it's just plugging numbers.

The recipe / formula

```
H = [ (R + i) x S ] / C
```

Where: - **H** = final Hadoop storage needed - **R** = Replication factor (usually 3, Hadoop keeps 3 copies of everything) - **i** = Intermediate data factor (temp space for Map/Reduce jobs, e.g. 0.30) - **S** = total data to store (raw + growth + job output) - **C** = Compression ratio (no compression = 1; here 2.5 means data shrinks 2.5x)

Number of DataNodes: **n = H / d** (d = usable disk per node)

Why (R + i)?

- You store the data **3 times** (R = 3) for safety.
- Plus extra scratch space for processing (i).
- So total multiplier before compression = (3 + 0.30) = 3.30.
- Then compression divides it down.

Worked Example - the 6200 TB problem

Given: Raw = 6200 TB, +300 TB growth, Output = 20% of available data, i = 0.30, C = 2.5, R = 3, each DataNode = 120 TB usable.

Part I - Total logical data after 6-month growth:

```
= Raw + Growth
= 6200 + 300
= 6500 TB
```

Part II - Effective Hadoop storage requirement:

Step 1, add the job output (20% of available data):

```
Output = 0.20 x 6500 = 1300 TB
S = 6500 + 1300 = 7800 TB   (total data to actually store)
```

Step 2, apply the formula:

```
H = [ (R + i) x S ] / C
H = [ (3 + 0.30) x 7800 ] / 2.5
H = [ 3.30 x 7800 ] / 2.5
H = 25740 / 2.5
H = 10296 TB
```

Part III - Storage expansion factor (vs Part I):

```
= H / (logical size from Part I)
= 10296 / 6500
= 1.58x  (about 1.58 times bigger)
```

Part IV - Minimum number of DataNodes:

```
n = H / d = 10296 / 120 = 85.8
then round UP, so 86 DataNodes
```

(Always round UP, you can't have 0.8 of a machine, and you need enough space.)

> Key reminders: Replication is usually **3**. Always **round node count UP**. Compression **divides** (makes it smaller). Output % is taken on the data you have *before* adding it.

NUMERICAL TYPE 3 - Amdahl's Law & Gustafson's Law

Q4 on your numerical sheet (the 32-node video analytics one). Two formulas, that's it.

The two formulas (MEMORIZE)

```
Amdahl's Law:     Speedup = 1 / ( s + p/N )
Gustafson's Law:  Speedup = s + N x p     (same as  N - s(N-1) )
```

Where: - **s** = serial fraction (the part that CAN'T be parallelized) - **p** = parallel fraction (the part that CAN). Note **p = 1 - s**. - **N** = number of processors/nodes

The difference in plain English

- **Amdahl** = "problem size is FIXED." No matter how many CPUs you add, the serial part holds you back. Speedup hits a ceiling (max = 1/s). Pessimistic.
- **Gustafson** = "problem GROWS as you add CPUs" (more nodes then bigger workload). The parallel part keeps growing so speedup keeps climbing. Optimistic and realistic for big data.

Worked Example - video analytics, 32 nodes

Stages given: | Stage | Fraction | Parallel? | |-------|----------|-----------| | Feature Extraction | 0.45 | 100% parallel, serial 0 | | Object Tracking | 0.35 | 20% serial, so serial = 0.20 x 0.35 = 0.07 | | Result Aggregation | 0.20 | 100% serial, serial 0.20 |

First find the serial fraction s:

```
s = 0 + 0.07 + 0.20 = 0.27
p = 1 - s = 0.73
N = 32
```

Q1 - Amdahl speedup:

```
Speedup = 1 / ( s + p/N )
        = 1 / ( 0.27 + 0.73/32 )
        = 1 / ( 0.27 + 0.0228 )
        = 1 / 0.2928
        = 3.42x
```

Q2 - Parallel fraction of the whole app:

```
p = 0.73, so 73%
```

Q3 - Gustafson speedup:

```
Speedup = s + N x p
        = 0.27 + 32 x 0.73
        = 0.27 + 23.36
        = 23.63x
```

Q4 - Compare and explain: - Amdahl gives only **3.42x** because it assumes the workload is fixed, the 27% serial part becomes the bottleneck and caps speedup (max possible = 1/0.27 which is about 3.7x). - Gustafson gives **23.63x** because it assumes the problem grows with more nodes, the parallel work scales up, so the serial part becomes a tiny fraction of the total. - **They differ purely because of the assumption:** fixed problem size (Amdahl) vs problem that scales with processors (Gustafson). For big-data systems that keep processing more frames, Gustafson is the more realistic view.

> Steps every time: (1) find serial fraction s by adding up the non-parallel bits, (2) p = 1 - s, (3) plug into both formulas.

PART B - THE THEORY (explained simple)

This covers the other 50%. I grouped it by topic. The starred ones are most likely from Assignments 3 & 4.

---

1. RPC (Remote Procedure Call) [important]

Big idea: RPC makes calling a function on a *remote* server feel exactly like calling a normal local function. You just write `result = doit(a, b)` and the magic happens behind the scenes. Goal: *make distributed computing look like centralized computing.*

How it works, the STUBS: - **Client stub** = fake local function on the client. It packs your arguments into a message (this packing = **marshalling**), sends it over the network, waits, unpacks the reply. - **Server stub (skeleton)** = receives the message, unpacks arguments (**unmarshalling**), calls the real function on the server, packs the result, sends it back. - You (the programmer) never see the networking. The stubs hide it all.

Marshalling = converting data into a standard byte format for network transfer (because different machines store bytes differently, **big endian** vs **little endian**). A neutral format like **XDR** fixes this.

Parameter passing problem: You can pass **by value** (a copy) over RPC, but **NOT by reference** (a pointer), because a pointer/address is only meaningful on the original machine; the server's memory is different.

Binding = how the client finds the server. The server registers itself with a **directory service / port mapper** (says "here's my address and the functions I offer"). The client asks the port mapper where the server is, gets the address, then connects.

Examples that use RPC: NFS (network file system), DCOM (Microsoft), CORBA (uses an ORB), Java RMI.

---

2. Sockets & ZeroMQ (HUGE for Assignment 3) [important]

Sockets = the basic communication endpoint (write data out, read data in). They're low-level and easy to mess up. TCP socket operations: `socket, bind, listen, accept, connect, send, receive, close`.

ZeroMQ (OMQ) = a higher-level, easier messaging library. The "zero" = **zero broker** (no dedicated middleman server needed). All communication is **asynchronous**. It pairs sockets together. **3 patterns:**

Pattern 1 - Request-Reply (REQ ↔ REP)

Like classic client-server. Client uses **REQ** socket, server uses **REP** socket. Client sends, then server replies. (This is the "client asks server for the time" question.)

Pattern 2 - Publish-Subscribe (PUB ↔ SUB)

Server **publishes** messages on topics; clients **subscribe** to topics they care about. Server uses **PUB**, clients use **SUB**. If no one subscribed to a message, it's **lost**. A client gets **nothing** until it explicitly subscribes. (This is the "weather broadcast" question.)

Pattern 3 - Pipeline (PUSH ↔ PULL)

A process **pushes** results out; any available worker **pulls** them in. The pusher doesn't care who pulls, first available worker takes it. Keeps as many workers busy as possible.

Code you should be ready to write:

ZeroMQ Request-Reply, TIME server (Assignment 4 / Ass4b Q3):

```
# ---------- SERVER ----------
import zmq, time
context = zmq.Context()              # create context
socket = context.socket(zmq.REP)     # REP = reply socket
socket.bind("tcp://127.0.0.1:5000")  # bind to address
while True:
    request = socket.recv_string()   # wait for a request
    if request == "TIME":
        socket.send_string(time.asctime())   # reply with current time
    else:
        socket.send_string("ERROR: invalid request")  # error for anything else

# ---------- CLIENT ----------
import zmq
context = zmq.Context()
socket = context.socket(zmq.REQ)     # REQ = request socket
socket.connect("tcp://127.0.0.1:5000")  # connect to server
socket.send_string("TIME")           # send the literal string TIME
reply = socket.recv_string()         # receive the reply
print(reply)                         # print the timestamp
```

ZeroMQ Publish-Subscribe, Weather (Assignment 3 Q1):

```
# ---------- PUBLISHER (server) ----------
import zmq, time
context = zmq.Context()
socket = context.socket(zmq.PUB)      # PUB socket
socket.bind("tcp://127.0.0.1:5000")
while True:
    socket.send_string("WEATHER Islamabad 32C")  # topic "WEATHER" + body
    time.sleep(3)                                  # every 3 seconds

# ---------- SUBSCRIBER (client) ----------
import zmq
context = zmq.Context()
socket = context.socket(zmq.SUB)      # SUB socket
socket.connect("tcp://127.0.0.1:5000")
socket.setsockopt_string(zmq.SUBSCRIBE, "WEATHER")  # subscribe ONLY to WEATHER
for i in range(5):                    # receive 5 messages then stop
    msg = socket.recv_string()
    print(msg)
```

---

3. MPI - Blocking vs Non-Blocking (HUGE for Assignments 3 & 4) [important]

MPI (Message Passing Interface) = built for *parallel* applications where lots of processes talk to each other intensively. Better than plain sockets for clusters (TCP/IP overhead is too high there). Each process gets a **rank** (unique ID: 0, 1, 2...) and knows the total **size** (number of processes). Run with: `mpiexec -n 4 python script.py` (starts 4 processes).

Blocking communication, `send` / `recv`

The program **waits/freezes** until the communication is done before moving on.

```
if rank == 0:
    comm.send(data, dest=1, tag=21)   # blocks until sent
elif rank == 1:
    data = comm.recv(source=0, tag=21)  # blocks until received
```

- Good: Simple, easy to understand, good for small programs.
- Bad: Process sits idle while waiting, so low performance in parallel systems.

Non-blocking communication, `isend` / `irecv`

Starts the communication and **immediately continues** doing other work. You check completion later with `.wait()`. (The `i` stands for "immediate.")

```
if rank == 0:
    req = comm.isend(data, dest=1, tag=21)  # starts sending, keeps going
    req.wait()                              # later: make sure it finished
elif rank == 1:
    req = comm.irecv(source=0, tag=21)
    data = req.wait()
```

- Good: Better parallelism and efficiency; can overlap communication with computation.
- Bad: More complex; you MUST sync with `.wait()` (or `.test()`).

"What if you remove `req.wait()` on the sender?" (exam favourite)

If you delete `req.wait()`: - The send may **not finish** before the program continues or exits. - Data may be **incomplete / still in progress** when the program ends. - Possible **race condition**, isend only *starts* the send; without `wait()` there's **no guarantee the message was actually delivered**. - Receiver might get **incomplete/corrupted data**, or wait **forever**. - **So `wait()` is necessary** to guarantee the message is safely transmitted and processes are properly synced.

Who sends / who receives? (they always ask this)

The process with **rank 0** prepares the data and **sends** it. The process with **rank 1 receives** it. Easy mark, just say it.

Other MPI ops to know: `bcast` (broadcast same data from root to all), `scatter` (split data across processes), `gather` (collect results back). `ssend` (synchronous send, waits until transmission starts), `bsend` (buffered send).

---

4. UMA vs NUMA vs SMP vs MPP (Assignment 4 / Ass4b Q1) [important]

This is a comparison question, make a table, you get full marks.

Architecture	Memory Access Time	Scalability	Programming	Use Case
UMA (Uniform Memory Access)	**Same** for all processors (shared central memory)	**Poor**, memory becomes a bottleneck as you add processors	**Easy** (one shared memory)	Personal computers, small SMP systems
NUMA (Non-Uniform Memory Access)	**Different**, local memory fast, remote memory slow	**Good**, scales to large systems	**Harder** (must manage local vs remote memory)	High-performance servers, supercomputers, big data centers
SMP (Symmetric Multiprocessing)	Equal (it's basically UMA-style: all CPUs share one memory & one OS)	Limited (shared bus/memory bottleneck)	Easy (single shared OS & memory)	Multi-core servers, desktops
MPP (Massively Parallel Processing)	Each node has its **own private memory** (nothing shared); nodes talk by messages	**Excellent**, thousands of nodes	Hard (message passing between nodes)	Huge data warehouses, large-scale analytics

One-liner to remember: - **UMA/SMP** = shared memory, equal access, simple but doesn't scale. - **NUMA** = each CPU has fast local + slow remote memory, scales better. - **MPP** = totally separate nodes, message-passing, scales massively but hardest to program.

---

5. Hadoop & HDFS (Storage) [important]

Hadoop = open-source framework to store and process **massive** data across thousands of cheap ("commodity") machines. Handles failures itself. 4 core parts: **HDFS** (storage), **YARN** (resource management), **MapReduce** (processing), **Hadoop Common** (libraries).

HDFS (Hadoop Distributed File System)

- Designed for **few, very large files** (avg > 500 MB), **NOT** lots of small files.
- Model: **Write Once, Read Often.** You can't edit a file, only **append** to the end.
- Files split into **blocks** (default **128 MB** in Hadoop 2.0, was 64 MB in Hadoop 1.0).
- Each block is **replicated 3 times** for fault tolerance.

Why HDFS for big files, not small files? (Assignment 4 Q1) - The **NameNode** keeps metadata for every file/block **in RAM**. Millions of tiny files = millions of metadata entries = NameNode runs out of memory. - Big files = fewer blocks = less metadata = efficient. Small files waste the NameNode's memory and create overhead. HDFS is built for huge sequential reads, not tiny scattered ones.

The daemons (the worker processes)

- **NameNode** = the boss / address book. Stores **metadata** (which blocks make up which file, and where each block lives). Does **NOT** store actual data. It's the **single point of failure** in Hadoop 1.0.

- **DataNode** = the workers that actually **store the data blocks**. Send "heartbeats" to the NameNode.
- **Secondary NameNode** = **NOT** a backup! It just does periodic **checkpoints** (merges the NameNode's edit logs with its image to keep things tidy). Default every 1 hour.
- Hadoop 2.0 fixed the single-point-of-failure with **High Availability**: an **Active** + a **Standby** NameNode, **JournalNode** (keeps them in sync), and **ZooKeeper** (coordination).

Rack Awareness (Assignment 4 Q2)

HDFS knows which physical **rack** each DataNode is in. When it places the 3 replicas: - 1 copy on the **current node**, - 1 copy on **another node in the same rack**, - 1 copy on a node in a **different rack**.

How it helps fault tolerance: If a whole rack fails (power/network), there's still a copy on a different rack, so no data lost. It also reduces cross-rack network traffic for reads. Smart placement = survives both node failures AND rack failures.

---

6. YARN (Yet Another Resource Negotiator) [important]

Why it exists: In Hadoop 1.0, the **JobTracker** did EVERYTHING (resource management + job scheduling) and got overloaded; it also only ran MapReduce and couldn't scale past about 4000 nodes. YARN **splits** the JobTracker's jobs into separate pieces, so Hadoop can run many types of processing (not just MapReduce).

YARN components: - **ResourceManager (RM)** = cluster-level boss. Allocates CPU & memory to applications. Only cares about scheduling. Runs on a dedicated master node. - **NodeManager (NM)** = one per node (slave). Launches containers, monitors resource usage, reports to RM. - **ApplicationMaster (AM)** = one **per application/job**. Negotiates resource containers from the RM, monitors the job's progress. Short-lived. - **Container** = a bundle of resources (CPU+memory) where a task actually runs.

Schedulers (how jobs get their turn): - **FIFO** = first-come-first-served (in submission order). - **Capacity Scheduler** = multiple tenants share the cluster; each queue gets guaranteed capacity; free resources can be borrowed. - **Fair Scheduler** = every job gets a fair share of resources over time.

One-line: YARN turned Hadoop from "only batch MapReduce" into a general platform that also runs Spark, Storm (streaming), Giraph (graphs), etc., all on one cluster.

---

7. MapReduce [important]

Big idea: A programming model to process huge data in parallel across many machines. Two phases: - **Map** = extract/transform something from each record, produces `<key, value>` pairs. - **Reduce** = aggregate/summarize all values with the same key.

It gives you automatic parallelization, load balancing, and **fault tolerance** (failed tasks just restart, because mappers/reducers are independent and deterministic).

Classic Word Count example: - Input: `"Dear Bear River Car Car River Deer Car Bear"` - **Map** output: `<Dear,1> <Bear,1> <River,1> <Car,1> <Car,1> ...` (each word maps to 1) - **Shuffle/sort** groups same keys together. - **Reduce** output: `<Dear,1> <Bear,2> <River,2> <Car,3> <Deer,1>` (sums per word)

Key terms: - **InputSplit** = logical chunk of data = one map task. - **RecordReader** = converts the split into `<key,value>` pairs for the mapper. - **Map input** default key = line offset; value = the line text. - Mapper's **intermediate output** is stored on the **local disk** of the mapper node (**NOT** in HDFS), and is **deleted** after the job finishes. (This is why HDFS storage math uses an "intermediate data factor.") - Classes: **Mapper, Reducer, Driver** (Driver sets up & configures the job).

---

8. Spark & Flink vs Hadoop MapReduce (Assignment 4 / Ass4b Q2) [important]

Apache Spark = open-source distributed framework for big-data analytics. Does batch, real-time streaming, machine learning, and graph processing. Works in many languages (Java, Scala, Python, R). Can run on Hadoop clusters.

Why Spark/Flink beat Hadoop MapReduce for iterative & streaming jobs: - Spark does **in-memory** computation, keeps intermediate data in RAM instead of writing to disk after every step. Hadoop MapReduce writes to disk repeatedly, so it's slow. - **Way less disk I/O**, so much faster, especially for **iterative** algorithms (machine learning, where the same data is processed many times) and **streaming** (real-time). - Hadoop MapReduce is mainly **batch** only; Spark/Flink handle **real-time streams**.

When is Hadoop MapReduce still fine? Large-scale **offline batch** jobs where real-time speed isn't critical, and low-cost storage/analytics.

One consistency / fault-tolerance / security concern (pick one to mention): - *Fault tolerance:* if a node crashes mid-job, partial results can be lost, frameworks use replication / lineage (Spark's RDDs recompute lost data) to recover. - *Consistency:* keeping replicated data in sync across nodes is hard (a read might hit a stale copy). - *Security:* data moving across many nodes opens up breach risk, needs encryption & authentication.

---

9. Clock Synchronization - Why & How (lighter theory)

Why sync clocks? Computers each have their own clock (unlike CPUs in one machine that share a clock). In distributed systems there's **no bound** on message/processing delays (asynchronous model). Out-of-sync clocks cause real bugs, e.g. logs showing a ticket "bought after the flight was full," or two gamers killing a target and not knowing who was first.

Time standards: TAI (atomic time), **UTC** (international standard, adds leap seconds), re-transmitted by **GPS satellites**.

Cristian's Algorithm: client asks a time server, server replies with its time, client adds **RTT/2** to account for travel. Simple but: single point of failure, accuracy depends on symmetric network delay.

NTP (Network Time Protocol): the powerful real-world version. Uses a **hierarchy of strata**: - **Stratum 0** = ultra-accurate reference clocks (atomic, GPS), not on the network. - **Stratum 1** = servers directly connected to Stratum 0 (primary servers). - **Stratum 2, 3...** = get time from higher strata (slightly less accurate each level). - Lower stratum number = closer to the true time. NTP talks to **multiple servers**, filters out bad ones ("falsetickers"), and averages the good ones ("truechimers"), so it's robust and continuous. (The delay/offset formulas in Numerical Type 1 come from NTP.)

---

10. Data Pipeline & Ingestion (lighter, skim this)

- **Data Pipeline** = the whole journey of data: ingest, store, process, analyze, visualize. Separates compute from storage.
- **ETL** (Extract, Transform, Load) is a **subset** of a data pipeline; ETL is usually **batch**, while a full pipeline does **batch + real-time**.
- **Processing models:** Batch (jobs run to completion, no user input, Hadoop MapReduce), Interactive/Online (needs user input, web apps), Stream (record-by-record, data in motion, sensors/IoT), Real-time (hard deadlines, avionics, nuclear), Parallel.
- **Ingestion challenges:** slow (data comes in many formats, must convert to common format like JSON), complex/expensive, vulnerable (data moving = security risk).
- **Ingestion tools: Apache Flume** (collects/aggregates streaming **log** data into HDFS; flow: Agent, Collector, Storage), **Apache NiFi** (easy GUI, routing & transformation).

PART C - THE EXACT ASSIGNMENT QUESTIONS (so nothing surprises you)

Assignment 3 (Faisal Hayat): - Q1: Write ZeroMQ **Publisher + Subscriber** for a weather alert system (topic "WEATHER"). See code in section B-2. - Q2: MPI image-metadata **blocking vs non-blocking**; who sends/receives; what happens if `req.wait()` is removed. See section B-3.

Assignment 4 (Faisal Hayat): - Q1: Why HDFS for **large files** not small files. Section B-5. - Q2: **Rack awareness** & how it helps fault tolerance. Section B-5. - Q3: Write **request-response** client/server code with comments. Section B-2 (REQ/REP).

Assignment 4 (SZABIST / Asma Niaz Awan, the theory + numerical heavy one): - Q1: **UMA / NUMA / SMP / MPP** comparison. Section B-4. - Q2: **Spark/Flink vs Hadoop MapReduce** + one concern. Section B-8. - Q3: ZeroMQ **request-reply TIME** server/client code. Section B-2. - Q4: MPI **sync vs async** dictionary; who sends/receives; explain difference. Section B-3.

Numerical sheet (your friend's tip, VERY likely): - NTP delay & offset (x2). Numerical Type 1. - HDFS storage planning (4 parts). Numerical Type 2. - Amdahl & Gustafson. Numerical Type 3.

PART D - 60-SECOND CHEAT SHEET (glance before you walk in)

Formulas:

```
NTP delay:    d = (t4-t1) - (t3-t2)
NTP offset:   o = [(t2-t1) + (t3-t4)] / 2     (+o means client behind)
Cristian:     correct time = T_server + RTT/2
HDFS:         H = (R + i) x S / C    ;   nodes = H / d  (round UP)
Amdahl:       Speedup = 1 / (s + p/N)
Gustafson:    Speedup = s + N x p
              (p = parallel, s = serial, p = 1-s)
```

One-liners: - RPC = remote call feels local; **stubs** marshal/unmarshal; can't pass by reference. - ZeroMQ patterns: **REQ/REP** (req-reply), **PUB/SUB** (publish-subscribe), **PUSH/PULL** (pipeline). Zero = zero broker. - MPI **send/recv** = blocking (waits); **isend/irecv** = non-blocking (continue, then `.wait()`). Remove wait() = data maybe not delivered / race condition. - Rank 0 sends, Rank 1 receives. - UMA/SMP = shared memory equal access (doesn't scale). NUMA = local fast/remote slow (scales). MPP = separate memory + messages (scales massively). - HDFS: NameNode = metadata in RAM (don't store data); DataNode = stores blocks; Secondary NameNode = checkpoints, NOT a backup. Block = 128 MB. Replication = 3. - Rack awareness: 1 copy same node, 1 same rack, 1 different rack, so it survives a rack failure. - YARN: ResourceManager (cluster boss) + NodeManager (per node) + ApplicationMaster (per job). Replaced overloaded JobTracker. - MapReduce: Map gives `<key,value>`; Reduce aggregates by key. Intermediate data on **local disk**, not HDFS. - Spark beats MapReduce because **in-memory** (less disk I/O), great for iterative ML & streaming. MapReduce = batch only.

You got this. Do the numericals first in the exam (guaranteed marks), then the theory. Good luck!